BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: C Batch: 2 Practical No: 9

Roll no: 56 PRN No: 12411305 Name of Student: Anuja Dhiraj Kumbhar

Subject Name : Deep Learning

Problem Statement :

Perform image classification using a pre-trained Vision Transformer (ViT) model and compare its performance with a CNN model.

Algorithm :

1. Start.
2. Import the required Python libraries, including PyTorch, torchvision, Hugging Face Transformers, scikit-learn, NumPy, and Matplotlib.
3. Load the CIFAR-10 dataset from Hugging Face.
4. Select 8,000 images for training, 1,000 images for validation, and 1,000 images for testing.
5. Define the ten image classes: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck.
6. Preprocess the images for the CNN by resizing them to 32 × 32 pixels and converting them into tensors.
7. Preprocess the images for ViT by resizing them to 224 × 224 pixels and normalizing them using the pre-trained model's image processor.
8. Create data loaders for training, validation, and testing.
9. Design a CNN model consisting of convolutional layers, ReLU activation, max-pooling, adaptive average pooling, and a fully connected output layer.
10. Load the pre-trained Vision Transformer model, `google/vit-base-patch16-224`, and modify its classification head to predict ten classes.
11. Freeze the pre-trained ViT backbone and train its classification head.
12. Train the CNN using the training dataset and evaluate it on the validation dataset.
13. Train the ViT classification head and evaluate the model on the validation dataset.
14. Measure the training time and inference time of both models.
15. Evaluate both models on the test dataset.

16. Calculate accuracy, precision, recall, and F1-score.
17. Compare the performance of CNN and ViT using a table and graphs.
18. Display sample predictions from both models.
19. Draw conclusions based on the experimental results.
20. Stop.

Code :

```
import time
import numpy as np
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
from datasets import load_dataset
from torchvision import transforms
from transformers import (
    AutoImageProcessor,
    ViTForImageClassification
)
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score
)

# Device configuration
device = torch.device(
    "cuda" if torch.cuda.is_available() else "cpu"
)
print("Device:", device)

# Load CIFAR-10 dataset
dataset = load_dataset("uoft-cs/cifar10")

train_data = dataset["train"].select(range(8000))
val_data = dataset["train"].select(range(8000, 9000))
test_data = dataset["test"].select(range(1000))

class_names = [
    "airplane", "automobile", "bird", "cat", "deer",
    "dog", "frog", "horse", "ship", "truck"
]
```

```
# CNN image preprocessing
cnn_transform = transforms.Compose([
    transforms.Resize((32, 32)),
    transforms.ToTensor()
])

# ViT image processor
processor = AutoImageProcessor.from_pretrained(
    "google/vit-base-patch16-224"
)

class CIFAR10Dataset(Dataset):
    def __init__(self, data, transform=None,
                 image_processor=None):
        self.data = data
        self.transform = transform
        self.image_processor = image_processor

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        item = self.data[idx]
        image = item["img"].convert("RGB")
        label = int(item["label"])

        if self.image_processor is not None:
            inputs = self.image_processor(
                images=image,
                return_tensors="pt"
            )
            image = inputs["pixel_values"].squeeze(0)
        else:
            image = self.transform(image)

        return image, label


# Create datasets and data loaders
cnn_train = CIFAR10Dataset(
    train_data, transform=cnn_transform
)
cnn_val = CIFAR10Dataset(
    val_data, transform=cnn_transform
)
```

```
cnn_test = CIFAR10Dataset(
    test_data, transform=cnn_transform
)

vit_train = CIFAR10Dataset(
    train_data, image_processor=processor
)
vit_val = CIFAR10Dataset(
    val_data, image_processor=processor
)
vit_test = CIFAR10Dataset(
    test_data, image_processor=processor
)

cnn_train_loader = DataLoader(
    cnn_train, batch_size=64, shuffle=True
)
cnn_val_loader = DataLoader(
    cnn_val, batch_size=64
)
cnn_test_loader = DataLoader(
    cnn_test, batch_size=64
)

vit_train_loader = DataLoader(
    vit_train, batch_size=16, shuffle=True
)
vit_val_loader = DataLoader(
    vit_val, batch_size=16
)
vit_test_loader = DataLoader(
    vit_test, batch_size=16
)


# Define CNN model
class CNN(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()

        self.features = nn.Sequential(
            nn.Conv2d(3, 32, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),
```

```
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),

            nn.Conv2d(64, 128, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.AdaptiveAvgPool2d((1, 1))
        )

        self.classifier = nn.Linear(128, num_classes)

    def forward(self, x):
        x = self.features(x)
        x = torch.flatten(x, 1)
        return self.classifier(x)


# Train CNN
def train_cnn(model, loader, epochs=5):
    model.to(device)
    optimizer = torch.optim.Adam(
        model.parameters(), lr=0.001
    )
    criterion = nn.CrossEntropyLoss()

    start = time.time()

    for epoch in range(epochs):
        model.train()
        total_loss = 0

        for images, labels in loader:
            images = images.to(device)
            labels = labels.to(device)

            optimizer.zero_grad()
            outputs = model(images)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()

            total_loss += loss.item()

        print(
            f"Epoch {epoch + 1}/{epochs}, "
```

```
            f"Loss: {total_loss / len(loader):.4f}"
        )

    return time.time() - start


# Define ViT model
vit = ViTForImageClassification.from_pretrained(
    "google/vit-base-patch16-224",
    num_labels=10,
    ignore_mismatched_sizes=True
)

# Freeze the pre-trained backbone
for param in vit.vit.parameters():
    param.requires_grad = False


# Train ViT classification head
def train_vit(model, loader, epochs=5):
    model.to(device)

    optimizer = torch.optim.AdamW(
        filter(lambda p: p.requires_grad,
               model.parameters()),
        lr=0.001
    )
    criterion = nn.CrossEntropyLoss()

    start = time.time()

    for epoch in range(epochs):
        model.train()
        total_loss = 0

        for images, labels in loader:
            images = images.to(device)
            labels = labels.to(device)

            optimizer.zero_grad()
            outputs = model(
                pixel_values=images
            ).logits

            loss = criterion(outputs, labels)
```

```
            loss.backward()
            optimizer.step()

            total_loss += loss.item()

        print(
            f"Epoch {epoch + 1}/{epochs}, "
            f"Loss: {total_loss / len(loader):.4f}"
        )

    return time.time() - start


# Evaluate model
def evaluate_model(model, loader, is_vit=False):
    model.eval()
    y_true = []
    y_pred = []

    start = time.time()

    with torch.no_grad():
        for images, labels in loader:
            images = images.to(device)

            if is_vit:
                outputs = model(
                    pixel_values=images
                ).logits
            else:
                outputs = model(images)

            predictions = outputs.argmax(dim=1)

            y_true.extend(labels.numpy())
            y_pred.extend(
                predictions.cpu().numpy()
            )

    inference_time = time.time() - start

    return {
        "Accuracy": accuracy_score(y_true, y_pred),
        "Precision": precision_score(
            y_true, y_pred, average="macro",
```

```
            zero_division=0
        ),
        "Recall": recall_score(
            y_true, y_pred, average="macro",
            zero_division=0
        ),
        "F1-Score": f1_score(
            y_true, y_pred, average="macro",
            zero_division=0
        ),
        "Inference Time (s)": inference_time
    }


# Initialize and train CNN
cnn = CNN()
cnn.to(device)

cnn_training_time = train_cnn(
    cnn, cnn_train_loader, epochs=5
)

# Initialize and train ViT
vit_training_time = train_vit(
    vit, vit_train_loader, epochs=5
)

# Evaluate both models
cnn_results = evaluate_model(
    cnn, cnn_test_loader
)

vit_results = evaluate_model(
    vit, vit_test_loader, is_vit=True
)

cnn_results["Training Time (s)"] = cnn_training_time
vit_results["Training Time (s)"] = vit_training_time

print("\nCNN Results:")
for key, value in cnn_results.items():
    print(f"{key}: {value}")

print("\nViT Results:")
for key, value in vit_results.items():
```

```
    print(f"{key}: {value}")


# Compare model performance
metrics = [
    "Accuracy", "Precision", "Recall", "F1-Score"
]

cnn_values = [
    cnn_results[m] * 100 for m in metrics
]
vit_values = [
    vit_results[m] * 100 for m in metrics
]

x = np.arange(len(metrics))
width = 0.35

plt.figure(figsize=(10, 5))
plt.bar(x - width / 2, cnn_values, width,
        label="CNN")
plt.bar(x + width / 2, vit_values, width,
        label="ViT")

plt.xticks(x, metrics)
plt.ylabel("Score (%)")
plt.title("CNN vs ViT Performance Comparison")
plt.legend()
plt.tight_layout()
plt.show()


# Display sample predictions
images, labels = next(iter(cnn_test_loader))
cnn.eval()

with torch.no_grad():
    predictions = cnn(
        images.to(device)
    ).argmax(dim=1).cpu()

plt.figure(figsize=(12, 6))

for i in range(min(8, len(images))):
    image = images[i].permute(1, 2, 0).numpy()
```

```
    plt.subplot(2, 4, i + 1)
    plt.imshow(image)
    plt.title(
        f"Actual: {class_names[labels[i]]}\n"
        f"Predicted: {class_names[predictions[i]]}"
    )
    plt.axis("off")

plt.tight_layout()
plt.show()
```

Output :

- **Dataset loading and preprocessing**

```
Training images: 8000
Validation images: 1000
Testing images: 1000
```

```
...  Datasets prepared successfully.
```

- **CNN model**

```
...  CNN(
       (features): Sequential(
         (0): Conv2d(3, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
         (1): ReLU()
         (2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
         (3): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
         (4): ReLU()
         (5): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
         (6): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
         (7): ReLU()
         (8): AdaptiveAvgPool2d(output_size=(1, 1))
       )
       (classifier): Linear(in_features=128, out_features=10, bias=True)
     )
```

- **Pre-trained Vision Transformer (ViT) Model Loading and Weight Initialization**

```
[transformers] You passed `num_labels=10` which is incompatible to the `id2label` map of length `1000`.
Loading weights: 100% ████████████████████████████████ 200/200 [00:00<00:00, 2054.25it/s]
[transformers] ViTForImageClassification LOAD REPORT from: google/vit-base-patch16-224
Key              | Status   |
-----------------+----------+-----------------------------------------------------------------------------------------
classifier.bias  | MISMATCH | Reinit due to size mismatch - ckpt: torch.Size([1000]) vs model:torch.Size([10])
classifier.weight | MISMATCH | Reinit due to size mismatch - ckpt: torch.Size([1000, 768]) vs model:torch.Size([10, 768])

Notes:
- MISMATCH: ckpt weights were loaded, but they did not match the original empty weight shapes.
Pre-trained ViT loaded successfully.
```

- **Training and Validation Accuracy of the Pre-trained ViT Model**

```
Epoch 1/5 | Loss: 0.2398 | Validation Accuracy: 95.00%
Epoch 2/5 | Loss: 0.0986 | Validation Accuracy: 95.70%
Epoch 3/5 | Loss: 0.0726 | Validation Accuracy: 96.20%
Epoch 4/5 | Loss: 0.0535 | Validation Accuracy: 95.70%
Epoch 5/5 | Loss: 0.0425 | Validation Accuracy: 95.70%
ViT training time: 596.22 seconds
```

- **Training and Validation Performance of the CNN Model**

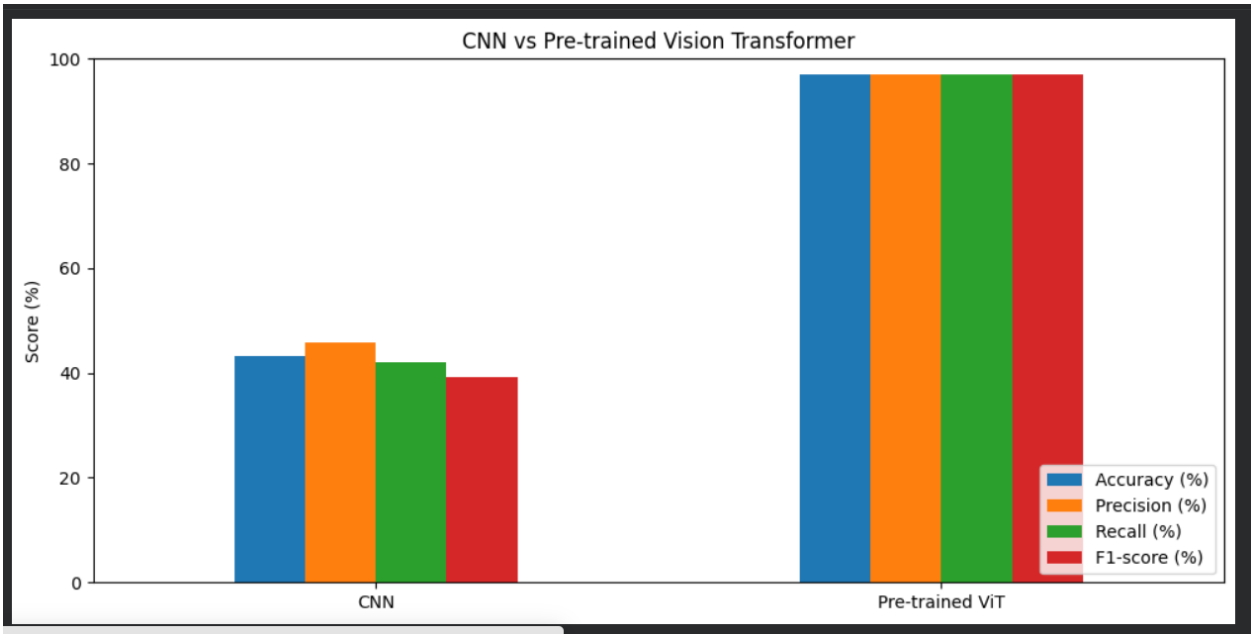
```
Epoch 1/5 | Loss: 2.0721 | Validation Accuracy: 27.80%
Epoch 2/5 | Loss: 1.8320 | Validation Accuracy: 32.50%
Epoch 3/5 | Loss: 1.7090 | Validation Accuracy: 35.40%
Epoch 4/5 | Loss: 1.6365 | Validation Accuracy: 42.40%
Epoch 5/5 | Loss: 1.5914 | Validation Accuracy: 41.10%

CNN training time: 9.48 seconds
```

- **Performance Comparison of CNN and Pre-trained ViT**

```
                 Accuracy (%)  Precision (%)  Recall (%)  F1-score (%)  \
CNN                      43.2          45.79       42.11         39.22
Pre-trained ViT          97.1          97.00       97.02         97.00

                 Inference time (s)  Training time (s)
CNN                            0.14               9.48
Pre-trained ViT               12.91             596.22
```



- **Sample CIFAR-10 Image Classification Results Using CNN and ViT**

